

UNIT 3: UNSUPERVISED LEARNING

3.1 Introduction to Clustering:

- Clustering or cluster analysis is a machine learning technique, which groups the un-labelled dataset.
- It can be defined as *"A way of grouping the data points into different clusters, consisting of similar data points. The objects with the possible similarities remain in a group that has less or no similarities with another group."*
- The clustering technique is commonly used for statistical data analysis.
- The clustering technique can be widely used in various tasks. Some most common uses of this technique are:
 - Market Segmentation
 - Statistical data analysis
 - Social network analysis
 - Image segmentation
 - Anomaly detection, etc.
- Apart from these general usages, it is used by the **Amazon** in its recommendation system to provide the recommendations as per the past search of products.
- **Netflix** also uses this technique to recommend the movies and web-series to its users as per the watch history.
- The below diagram explains the working of the clustering algorithm. We can see the different fruits are divided into several groups with similar properties.

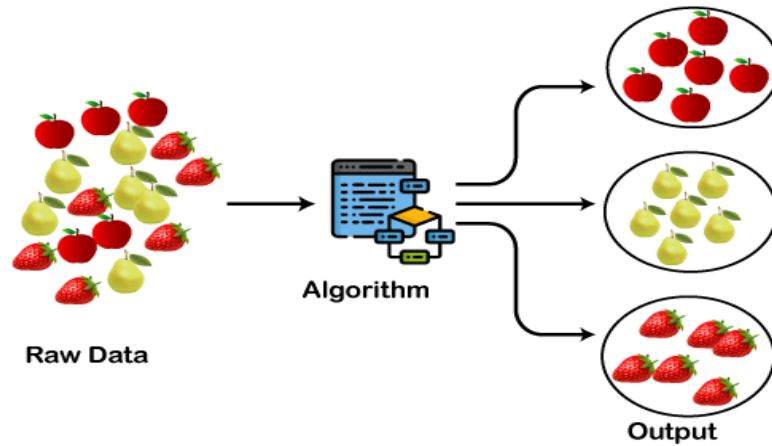


Fig: Clustering algorithms

Types of Clustering Methods

The clustering methods are broadly divided into **hard clustering** (data point belongs to only one group) and **soft clustering** (data points can belong to another group also). But there are also other various approaches of clustering exist. Below are the main clustering methods used in Machine learning:

1. Partitioning clustering- centroid-based method. eg; K-Means clustering algorithm.
2. Density-based clustering-varying densities and high dimensions.
3. Distribution model-based clustering- uses Gaussian mixture models. eg; Expectation-maximization clustering algorithm.
4. Hierarchical clustering: clusters to create a tree-like structure (dendrogram). eg; Agglomerative Hierarchical algorithm.
5. Fuzzy clustering- dataset has a set of membership coefficients, Fuzzy k-means algorithm.

3.2 K-Means Clustering:

- K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science. Which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on. It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.
- It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.
- It is a centroid-based algorithm, where each cluster is associated with a centroid.
- The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.
- This algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters and repeats the process until it does not find the best clusters.
- The value of k should be predetermined in this algorithm.

The k-means clustering algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.
 - Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.
- Hence each cluster has data points with some commonalities, and it is away from other clusters. The below diagram explains the working of the K-means clustering algorithm:

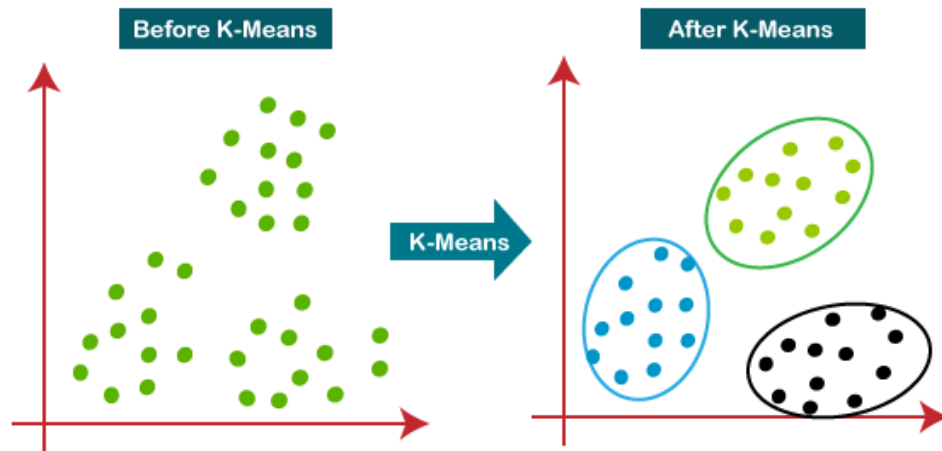


Fig: K-Means Algorithm

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each data point to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

Python Implementation of K-means clustering algorithm

The steps to be followed for the implementation are given below:

- **Data Pre-processing** - The first step will be the data pre-processing but for the clustering problem, it will be different from other models. Let's discuss it:
- **Importing Libraries**- Firstly, we will import the libraries for our model, which is part of data pre-processing. The code is given below:

```
# importing libraries
```

```
import numpy as np
import matplotlib.pyplot as mtp
import pandas as pd
```

In the above code, the **numpy** we have imported for the performing mathematics calculation, **matplotlib** is for plotting the graph, and **pandas** are for managing the dataset.

- **Importing the Dataset-** Next, we will import the dataset that we need to use. So here, we are using the iris_data.csv dataset. It can be imported using the below code:

```
data = pd.read_csv("give a downloaded dataset location ")
Eg; >>data=pd.read_csv("C:\\Users\\user\\Desktop\\Weka\\iris_csv.csv")
```

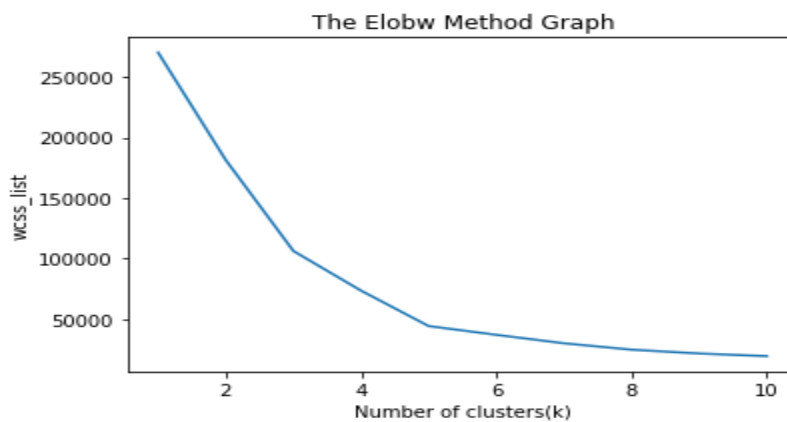
- **Finding the optimal number of clusters using the elbow method-** In the second step, we will try to find the optimal number of clusters for our clustering problem. So, as discussed above, here we are going to use the elbow method for this purpose. As we know, the elbow method uses the WCSS concept to draw the plot by plotting WCSS values on the Y-axis and the number of clusters on the X-axis. So we are going to calculate the value for WCSS for different k values ranging from 1 to 10. Below is the code for it:

#Finding optimal number of clusters using the Elbow method

```
from sklearn.cluster import KMeans
wcss_list= [] #Initializing the list for the values of WCSS
#Using for loop for iterations from 1 to 10.
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state= 42)
    kmeans.fit(x)
```

```
wcss_list.append(kmeans.inertia_)
mtp.plot(range(1, 11), wcss_list)
mtp.title('The Elbow Method Graph')
mtp.xlabel('Number of clusters(k)')
mtp.ylabel('wcss_list')
mtp.show()
```

Output: After executing the above code, we will get the below output:



- **Training the K-means algorithm on the training dataset-** As we have got the number of clusters, so we can now train the model on the dataset. To train the model, we will use the same two lines of code as we have used in the above section, but here instead of using `i`, we will use `5`, as we know there are 5 clusters that need to be formed. The code is given below:

#Training the K-means model on a dataset

```
kmeans = KMeans(n_clusters=5, init='k-means++', random_state= 42)
y_predict= kmeans.fit_predict(x)
```

- **Visualizing the clusters-** The last step is to visualize the clusters. As we have 5 clusters for our model, so we will visualize each cluster one by one. To visualize the clusters will use scatter plot using `mtp.scatter()` function of matplotlib.

#Visulaizing the clusters

```
mtp.scatter(x[y_predict == 0, 0], x[y_predict == 0, 1], s = 100, c = 'blue', label = 'Cluster 1') #for first cluster
mtp.scatter(x[y_predict == 1, 0], x[y_predict == 1, 1], s = 100, c = 'green', label = 'Cluster 2') #for second cluster
mtp.scatter(x[y_predict == 2, 0], x[y_predict == 2, 1], s = 100, c = 'red', label = 'Cluster 3') #for third cluster
mtp.scatter(x[y_predict == 3, 0], x[y_predict == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4') #for fourth cluster
mtp.scatter(x[y_predict == 4, 0], x[y_predict == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5') #for fifth cluster
mtp.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s = 300, c = 'yellow', label = 'Centroid')
mtp.title('Clusters of customers')
mtp.xlabel('Annual Income (k$)')
mtp.ylabel('Spending Score (1-100)')
mtp.legend()
mtp.show()
```

Output:



The output image is clearly showing the five different clusters with different colors.

Applications of K-Means clustering

K-Means clustering is widely used across various industries to group similar data points and discover patterns. Here are some key real-world applications:

1. Customer Segmentation (Marketing & Retail)

- Companies use K-Means to segment customers based on purchasing behavior, demographics, or preferences.
- Helps in targeted advertising and personalized recommendations.
- Example: Online retailers use it to group users into categories like frequent shoppers, bargain hunters, or luxury buyers.

2. Image Compression & Segmentation (Computer Vision)

- K-Means helps reduce image colors by clustering similar pixel values, making image storage more efficient.
- Also used in medical imaging for tumor detection by segmenting affected areas.

3. Anomaly Detection (Cybersecurity & Fraud Prevention)

- Identifies unusual patterns in network traffic to detect cyber threats.

- In banking, K-Means clusters normal transactions, making anomalies (potential fraud) stand out.

4. Document Clustering (Search Engines & NLP)

- Search engines use K-Means to group documents based on topic similarity.
- Helps in organizing large text datasets and improving search results.

5. Recommendation Systems (Streaming Platforms & E-commerce)

- Platforms like Netflix and Spotify group users with similar preferences for better content suggestions.
- E-commerce sites use it to recommend products by clustering users based on browsing history.

6. Biological & Medical Research (Genetics & Drug Discovery)

- Used to cluster genes with similar characteristics in genomic research.
- Helps in disease classification, identifying cancer types, and drug development.

7. Geospatial Clustering (Urban Planning & Logistics)

- Helps city planners group areas with similar infrastructure needs.
- Used in logistics to optimize delivery routes by clustering locations.

8. Social Media Analytics (User Behavior Analysis)

- Categorizes social media users based on interests, engagement levels, and activity patterns.
- Helps brands tailor content to different audience segments.

Example1: Simple numerical example of K-Means clustering:- Imagine we have the following data points in a two-dimensional space:

Data Point	X	Y
A	1	1
B	2	1
C	4	3
D	5	4

Let's assume we want to classify them into **two clusters** ($K = 2$).

Step 1: Initialize centroids

We randomly select two initial cluster centers:

- Centroid 1 at (1,1)
- Centroid 2 at (5,4)

Step 2: Assign points to the nearest centroid

Using **Euclidean distance**, we compute distances and assign each point to the nearest centroid:

- A (1,1) $\rightarrow$ closer to Centroid 1
- B (2,1) $\rightarrow$ closer to Centroid 1
- C (4,3) $\rightarrow$ closer to Centroid 2
- D (5,4) $\rightarrow$ closer to Centroid 2

New clusters:

- Cluster 1: {A, B}
- Cluster 2: {C, D}

Step 3: Update centroids

Now, we compute the new centroid positions:

- New Centroid 1 = Average of (A, B) = $(1+2)/2$, $(1+1)/2$ = **(1.5, 1)**

- New Centroid 2 = Average of (C, D) = $(4+5)/2$, $(3+4)/2$ = **(4.5,3.5)**

Step 4: Repeat until centroids stabilize

We reassign points based on the updated centroids and repeat until cluster centers do not change. This process continues iteratively, refining the clusters.

Example 2: Clustering customer purchases: Imagine we have a dataset of customers based on their spending habits:

Customer	Electronics	Clothing	Groceries
A	300	50	200
B	250	80	150
C	100	400	300
D	90	450	280
E	200	60	180
F	110	420	310

We want to divide these customers into **K = 2 clusters** based on spending patterns.

Step 1: Initialize Centroids

Randomly selecting two customers as initial centroids:

- **Centroid 1:** Customer A (300, 50, 200)
- **Centroid 2:** Customer D (90, 450, 280)

Step 2: Compute Distance & Assign Points

Using **Euclidean distance**, we assign each customer to the closest centroid.

After distance calculation:

- Customers {A, B, E} are closer to Centroid 1

- Customers {C, D, F} are closer to Centroid 2

Step 3: Update Centroids

Computing new centroids:

- **New Centroid 1 (Cluster 1 average):**
 $(300+250+200)/3, (50+80+60)/3, (200+150+180)/3$
 $= (250, 63.3, 176.7)$
- **New Centroid 2 (Cluster 2 average):**
 $(100+90+110)/3, (400+450+420)/3, (300+280+310)/3$
 $= (100, 423.3, 296.7)$

Step 4: Repeat Until Centroids Stabilize

We reassign points based on updated centroids and continue iterating until clusters remain unchanged.

Insights:

- Cluster 1: Customers who spend more on **electronics** and moderate amounts on groceries.
- Cluster 2: Customers who spend more on **clothing** with slightly higher grocery spending.

3.3 K-mode clustering:

- K-mode clustering is an unsupervised machine-learning technique used to group a set of data objects into a specified number of clusters, based on their categorical attributes. The algorithm is called “K-Mode” because it uses modes (i.e. the most frequent values) instead of means or medians to represent the clusters. K-modes is an algorithm for clustering categorical data. It is used to partition a dataset into a specified number of clusters, where each cluster is

characterized by a mode, which is the most frequent categorical value in the cluster.

- K-Modes is a clustering algorithm used in data science to group similar data points into clusters based on their categorical attributes. Unlike traditional clustering algorithms that use distance metrics, K-Modes works by identifying the modes or most frequent values within each cluster to determine its centroid. K-Modes is ideal for clustering categorical data such as customer demographics, market segments, or survey responses. It is a powerful tool for data analysts and scientists to gain insights into their data and make informed decisions.

K-Modes Algorithm

1. Pick K observations at random and use them as leaders/clusters.
2. Calculate the dissimilarities and assign each observation to its closest cluster.
3. Define new modes for the clusters.
4. Repeat 2-3 steps until there is no re-assignment required.

Applications of K-mode clustering in real life.

K-modes clustering is widely used in real-world scenarios where categorical data plays a central role. Here are some key applications:

i) Customer segmentation: Businesses use K-modes clustering to group customers based on preferences, demographics, and purchasing behaviors. This helps in targeted marketing, personalized recommendations, and improving customer experience.

ii) Medical diagnosis: In healthcare, K-modes clustering assists in identifying patient groups with similar symptoms, medical histories, and genetic factors. This aids in disease prediction, treatment planning, and personalized medicine.

iii) Fraud detection: Banks and financial institutions use K-modes clustering to detect fraudulent activities by clustering transactions based on categorical attributes like payment methods, locations, and types of purchases.

iv) Social media analysis: Platforms group users based on categorical attributes like interests, activity patterns, and content interactions. This helps in content recommendation, user engagement, and behavioral predictions.

v) Crime pattern analysis: Law enforcement agencies use K-modes clustering to categorize crime incidents based on location, type, time, and suspect details. This assists in predictive policing and resource allocation.

vi) Product categorization: E-commerce platforms classify products based on categorical attributes like brand, type, and customer ratings. This enhances search optimization and recommendation systems.

Implementation of the k-mode clustering algorithm in Python:

```
# importing necessary libraries
import pandas as pd
import numpy as np
# !pip install kmodes
from kmodes.kmodes import KModes
import matplotlib.pyplot as plt
%matplotlib inline
# Elbow curve to find optimal K
cost = []
K = range(1,5)
for k in list(K):
    kmode = KModes(n_clusters=k, init = "random", n_init = 5, verbose=1)
    kmode.fit_predict(data)
```

```
cost.append(kmode.cost_)  
plt.plot(K, cost, 'x-')  
plt.xlabel('No. of clusters')  
plt.ylabel('Cost')  
plt.title('Elbow Curve')  
plt.show()
```